

ASSIGNMENT 1 SOLUTIONS: LOOP INVARIANTS

Instructor: Orhan Özgüner

Due: September 12 before 11:59 PM

Problem 1

The greatest common divisor (GCD) of two integers a and b is defined as the largest integer that can divide both a and b without a remainder. For example, the GCD of 30 and 54 is 6, whereas the GCD of 7 and 5 is 1. The following procedure was developed by Euclid to compute the greatest common divisor of two positive integers a and b . In this exercise, we will prove the correctness of this algorithm.

```
procedure EUCLIDEAN( $a, b$ )
1   $x \leftarrow a$ 
2   $y \leftarrow b$ 
3  while  $x \neq y$  do
4      if  $x > y$  then
5           $x \leftarrow x - y$ 
6      else
7           $y \leftarrow y - x$ 
8  return  $x$ 
```

- (a) State the loop invariant for the while loop in this procedure.
- (b) Prove the loop invariant.
- (c) Prove that procedure EUCLIDEAN always terminates provided that a and b are positive integers.
- (d) Using the termination property of your loop invariant, prove that procedure EUCLIDEAN computes and returns the greatest common divisor of a and b .

Problem 1: Solution

a) Loop Invariant: At the beginning of each iteration of the loop, the GCD of the current values x and y remains equal to the GCD of the original inputs a and b . $\gcd(x, y) = \gcd(a, b)$

b) Loop Invariant Proof:

Initialization: Initially, $x = a$ and $y = b$. Substituting these values into the loop invariant gives, $\gcd(x, y) = \gcd(a, b)$, which clearly holds. Therefore, the loop invariant is true at the start of the loop.

Maintenance: Assume that Loop Invariant is true at the beginning of each iteration, $\gcd(x, y) = \gcd(a, b)$. In each iteration, there are three cases we need to consider:

- $x > y$

Proof. At the end of the loop, x is updated such that $x_{next} = x - y$. Due to the loop invariant of the previous iteration, $\gcd(x, y) = \gcd(a, b)$. Thus, we only need to show that $\gcd(x_{next}, y) = \gcd(x, y)$. Let $k = \gcd(x, y)$. Then, $x = kc_x$ and $y = kc_y$ for some integers c_x and c_y such that $c_x > c_y$ and $\gcd(c_x, c_y) = 1$. Thus, $x_{next} = x - y = kc_x - kc_y = k(c_x - c_y) = kc_{next}$ for some $c_{next} > 0$ such that $\gcd(c_{next}, c_y) = 1$. Therefore, $\gcd(x_{next}, y) = k = \gcd(x, y)$. $\square$

- $x < y$

Proof. The proof is the same as previous case when the places of x and y are swapped. $\square$

- $x = y$

Proof. The loop terminates without updating x or y . Since x or y values are not updated, then $\gcd(x, y) = \gcd(a, b)$. $\square$

c) Loop Termination Proof:

Given that a and b are positive integers, x and y are initially positive integers. To show that the loop terminates, we need to show (1) x or y is decreasing at each iteration, and (2) x and y remain positive integers. It is evident that $x \leftarrow x - y$ and $y \leftarrow y - x$ operations decrease x or y since x and y are both positive. It is also evident that x and y remain positive since x or y are always updated to $\max(x, y) - \min(x, y) > 0$ when $x \neq y$. Therefore, this loop always terminates.

d) Termination: After at most $x + y - 1$ number of iterations, the loop will terminate when $x = y$. To prove that the procedure returns the correct result, we apply the termination condition to the loop invariant. Given the loop invariant, $\gcd(x, y) = \gcd(a, b)$, the algorithm returns $\gcd(a, b) = \gcd(x, y) = \gcd(x, x) = x$. Thus, the algorithm correctly returns $\gcd(a, b)$.

Problem 2

We are given an array A consisting of n numbers sorted in increasing order. We are also given a target number t and we would like to find and return the index i such that $A[i] = t$. If A does not contain t , we would like to know that as well. Your algorithm should return -1 if t is not in the array. Write an efficient algorithm to solve this problem.

- Using pseudo-code, describe an algorithm that correctly solves this problem efficiently.
- Using loop invariants, prove that your algorithm is correct.

Problem 2: Solution

a) Pseudocode:

Algorithm 1 Binary Search

Require: A : a sorted array of integers

Require: t : the target integer

Ensure: The index of t in A or -1 if t is not present

```
1: procedure BINARY_SEARCH( $A, t$ )
2:    $low \leftarrow 1$ 
3:    $high \leftarrow \text{length}(A)$ 
4:   while  $low \leq high$  do
5:      $mid \leftarrow \lfloor \frac{low+high}{2} \rfloor$ 
6:     if  $A[mid] = T$  then
7:       return  $mid$ 
8:     else if  $A[mid] < T$  then
9:        $low \leftarrow mid + 1$ 
10:    else
11:       $high \leftarrow mid - 1$ 
12:    end if
13:  end while
14:  return  $-1$ 
15: end procedure
```

b) Proof of correctness

Loop Invariant: The target value t , if it exists, is within $A[low...high]$.

- **Initialization:** Before the loop starts, low is initialized to 1 (the start of the array), and $high$ is initialized to n . The entire array is the initial search space, so if t exists in the array, it must be within $A[low...high]$. Therefore, the loop invariant is true at the start of the loop.
- **Maintenance:** Assume the Loop Invariant holds for some iteration. Then, we find the middle element by taking the average of low and $high$. Then, we have three cases to consider:

– $A[mid] = t$

Proof. We return m and the algorithm has found the target value. □

– $A[mid] < t$

Proof. Since $A[mid] < t$ and the array is sorted then it must be that $i > mid$ if t is in A . Our algorithm in this case searches the new range $[mid + 1...high]$. Therefore, if the target value t exists, it is within $A[low...high]$ (note that $low = mid + 1$). Therefore, the loop invariant still holds. □

– $A[mid] > t$

Proof. Since $A[mid] > t$ and the array is sorted then it must be that $i < mid$ if t is in A . Our algorithm in this case searches the new range $[low...mid - 1]$. Therefore, if the target value t exists, it is within $A[low...high]$ (note that $high = mid + 1$). Therefore, the loop invariant still holds. $\square$

- **Termination:** The loop continues until (1) the target value t is found, or (2) $low > high$. In each iteration of the loop, the search interval defined by low and $high$ strictly decreases, either by moving low up or $high$ down. Since the array has a finite length, this shrinking interval guarantees that the loop will eventually terminate when $low > high$.

If (1) is satisfied, then we found a solution, thus, the algorithm terminates correctly finding the correct result. If (2) is satisfied, low exceeds $high$, it means that the indices have crossed each other, and the search space is invalid. Thus, the target value does not exist. Thus, the algorithm accurately returns -1 .

Problem 3

Two species are sharing planet Kepler-442b: Pisidians and Lydians. Members of either species roam individually. They can run into each other only in pairs (randomly) and whenever any two individuals run into each other, they fight. Since Pisidians are stronger and Lydians have magical powers, the outcome of a fight is always the same:

- If two Lydians fight, one of the Lydians dies and the other survives.
- If two Pisidians fight, both Pisidians die and a new Lydian comes to existence.
- If a Pisidian and a Lydian fight, the Lydian dies and the Pisidian survives.

At the beginning of time, there were m Lydians and n Pisidians on Kepler-442b. Observing that there will be only one individual left on Kepler-442b at eternity, identify the species of the eternal individual as a function of m and/or n . (*Hint:* Describe the process using a loop by writing a pseudocode, define a loop invariant, and prove the loop invariant)

Problem 3: Solution

Algorithm 1

Require: Two integers m and n which are the number of Lydians and Pisidians respectively.

Ensure: The species of the eternal individual: **Lydian** or **Pisidian**.

```
1:  $m' \leftarrow m$ 
2:  $n' \leftarrow n$ 
3: while  $m' + n' > 1$  do
4:   if Two Pisidians fight then
5:      $n' \leftarrow n' - 2$ 
6:      $m' \leftarrow m' + 1$ 
7:   else
8:      $m' \leftarrow m' - 1$ 
9:   end if
10: end while
11: if  $n' = 0$  then
12:   return Lydian
13: else
14:   return Pisidian
15: end if
```

Loop Invariant: Let k denote the number of individuals such that $k = m' + n'$. The number of individuals k decreases by one in each iteration, and the parity (evenness/oddness) of the number of Pisidians (n') remains unchanged: $n' \equiv n \pmod{2}$.

- **Initialization:** Initially, $n' = n$. Thus, $n' \equiv n \pmod{2}$. Therefore, the loop invariant is true at the start of the loop.
- **Maintenance:** Assume the Loop Invariant holds for some iteration. When two individuals fight, there are two cases we need to consider:
 - Two Pisidians fight

Proof. Both n' and m' will be updated such that $n_{next} = n' - 2$ and $m_{next} = m' + 1$. Thus, $k_{next} = n_{next} + m_{next} = m' + n' - 1 = k - 1$. Hence, (1) is satisfied. Due to the previous iteration of loop invariant, $n' \equiv n \pmod{2}$. Thus, to prove (2), we only need to show that $n_{next} \equiv n' \pmod{2}$ which is evident since $n_{next} = n' - 2$. Hence, (2) is satisfied as well. $\square$

- Two Pisidians do not fight

Proof. Only m' will be updated such that $m_{next} = m' - 1$. It is evident that (1) is satisfied since m' decreases by one. Moreover, it is evident (2) is satisfied since n' is not updated. $\square$

- **Termination:** Each iteration of the loop reduces the total number of individuals $m' + n'$ by exactly 1:

- - If two Pisidians fight: $n' \leftarrow n' - 2$, $m' \leftarrow m' + 1$, where net change is -1 .
- - If a Lydian and a Pisidian fight: $m' \leftarrow m' - 1$, where net change is -1 .

Since $m' + n'$ starts as a finite positive integer (equal to $m + n$), and decreases by 1 in each iteration, the loop must eventually reach the point where $m' + n' = 1$. Therefore, this loop always terminates.

Due to loop invariant (1), the loop will terminate after $k - 1$ iterations when k becomes 1. Due to loop invariant (2), the parity of n will be preserved. Thus, if $n \equiv 0 \pmod{2}$, then a Lydian will survive. Otherwise, a Pisidian will survive.